

180DC ML Team Dev Tasks

General Guidelines

- Complete the task described below: **Solve the Case — An Interactive Agentic RAG Investigation.**
- Deadline: 13th September, 12PM (noon)
- AI-powered coding tools are allowed and encouraged where they help accelerate development.
- You should understand the code you submit and be able to explain your architecture, implementation, dependencies, and technical decisions.
- You must select a suitable public dataset/corpus yourself. The corpus should contain enough interconnected information to support a multi-document investigation.

Submission Guidelines

- **Submission:** Submit your GitHub repository link through the submission form. The repository should contain the complete source code, preprocessing code, and required files to run the project.
- **Bonus Features:** Briefly describe any additional bonus features implemented.
- **AI Tools:** Briefly mention the AI coding tools used and how they were used.
- **Video Demo:** Create a 7–10 minute video demonstrating the application, including the core investigation flow, Investigator/Fact-Checker agent execution, key architectural decisions, and any bonus features.

Task: Solve the Case — An Interactive Agentic RAG Investigation

Type: AI / ML Application

Overview

Build an interactive detective-style AI system where the user investigates a case with the help of AI agents.

The system should analyze a corpus of documents, identify candidates, entities, events, relationships, and evidence, and build an evidence graph.

Two agents assist the investigation:

- **Investigator Agent** — forms theories, retrieves evidence, and self-corrects when evidence is insufficient.
- **Fact-Checker Agent** — actively searches for evidence that contradicts the Investigator's current theory.

The user can interrogate candidates, search evidence, inspect the investigation, make predictions, and submit a final verdict.

You must select a suitable public dataset/corpus yourself. The corpus should contain enough interconnected information to support multi-document investigation.

1. Backend — Mandatory

Implement a backend using FastAPI, Flask, or equivalent.

Suggested APIs:

POST /ingest	-> Process the selected corpus.
POST /interrogate	-> Chat with a candidate/suspect.
POST /search_evidence	-> Search the evidence corpus.
POST /investigate	-> Run the Investigator Agent.

POST /fact_check	-> Challenge the Investigator's theory.
POST /submit_verdict	-> Evaluate the user's final conclusion.

The exact API design is up to you.

2. Corpus Analysis & Evidence Graph — Mandatory

The Investigator should extract from the corpus:

- Candidates / suspects
- People, places, objects, events, timestamps, etc.
- Relationships between entities
- Evidence associated with documents

Represent these relationships as a graph. A lightweight implementation using networkx is sufficient.

3. RAG Layer — Mandatory

Implement retrieval over the selected corpus. The system should support:

- Keyword / lexical retrieval
- Semantic retrieval
- Combining the two is encouraged

A vector database or RAG framework is not required.

4. Investigator Agent — Mandatory

The Investigator should:

1. Form an initial theory.
2. Retrieve supporting evidence.
3. Evaluate whether the evidence is sufficient.
4. Reformulate the query and retry if necessary.
5. Return a grounded theory with citations.

Limit retries to avoid infinite loops.

5. Fact-Checker Agent — Mandatory

The Fact-Checker should deliberately search for evidence against the Investigator's theory. It should look for:

- Contradictions
- Alternative explanations
- Conflicting timelines
- Evidence supporting another candidate
- Weaknesses in the current theory

6. User Interaction — Mandatory

The frontend should allow users to:

- Interrogate candidates freely.
- Search the evidence corpus.
- Inspect the evidence graph.
- Make a prediction before seeing the complete agent results.

- Submit a final suspect/candidate and supporting evidence.

7. Structured Input / Output — Mandatory

Use structured JSON between major components. Example Pydantic schema:

```
from pydantic import BaseModel
from typing import List

class Citation(BaseModel):
    document_id: str
    claim: str

class InvestigationResult(BaseModel):
    theory: str
    confidence: float
    citations: List[Citation]
    needs_more_evidence: bool
```

Your schemas may differ depending on your implementation.

8. Grounding & Reliability — Mandatory

- Agent claims should reference document IDs.
- Citations must correspond to retrieved documents.
- The system should distinguish verified and unverified evidence.
- Include at least one misleading/unverified piece of information in your investigation.
- The Investigator should not confidently treat unverified information as established evidence.

9. Frontend — Mandatory

Build a simple interactive UI using Streamlit, React, or equivalent. It should contain:

- Candidate interrogation panel
- Evidence search
- Evidence/investigation board
- Investigator results
- Fact-Checker results
- Final verdict submission

Deployment — Mandatory

- Deploy the application so it is live and publicly accessible over the internet.
- AWS, Azure, GCP, or another suitable cloud/VPS provider may be used.
- Provide the live application URL in your submission.

Deliverables

1. **Backend API**
2. **Interactive frontend**
3. **Public dataset/corpus + preprocessing code**

4. **Investigator + Fact-Checker agents**
5. **1–2 page technical report** explaining your approach

Evaluation

- **Functionality** — Complete investigation works end-to-end.
- **Agentic behavior** — Self-correction and adversarial retrieval are genuinely implemented.
- **RAG quality** — Relevant evidence is retrieved effectively.
- **Corpus understanding** — Useful candidates, entities, and relationships are extracted.
- **Hallucination resistance** — Unverified evidence is not incorrectly treated as fact.
- **User interaction** — Users meaningfully participate in the investigation.

Bonus Features

- Advanced reranking
- Graph-aware retrieval
- Contradiction detection
- Temporal reasoning
- Multiple cases
- Persistent agent memory
- Parallel retrieval
- Agent execution traces
- Automatic faithfulness evaluation
- Visual evidence graph

What We Value

A smaller, well-engineered solution is preferable to a large collection of superficial features. We value sound engineering decisions, secure architecture, meaningful agentic AI integration, clear reasoning, and the ability to explain the system you built.